38.3 用量追踪

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~270k tokens，输出 ~5k tokens（本节）

LLM API 调用的费用是 AI 应用运营的主要成本之一。OpenClaw 提供了多层次的用量追踪系统，从单次请求的 token 计数到跨会话的成本汇总，帮助用户了解和控制开支。

38.3.1 Token 用量归一化

不同的 LLM 提供商（Anthropic、OpenAI、Google 等）返回的用量字段名称各不相同。 `normalizeUsage()` （ `src/agents/usage.ts` ，106 行）负责将各种格式统一为标准形式：

```
export type UsageLike = {
  input?: number;                // Anthropic 格式
  inputTokens?: number;          // 驼峰格式
  input_tokens?: number;         // 下划线格式
  promptTokens?: number;         // OpenAI 格式
  prompt_tokens?: number;        // OpenAI 下划线格式
  // ... output / cacheRead / cacheWrite 同样多种命名
};

export type NormalizedUsage = {
  input?: number;
  output?: number;
  cacheRead?: number;
  cacheWrite?: number;
  total?: number;
};

export function normalizeUsage(raw?: UsageLike | null): NormalizedUsage | undefined {
  const input = raw.input ?? raw.inputTokens ?? raw.input_tokens
    ?? raw.promptTokens ?? raw.prompt_tokens;
  const output = raw.output ?? raw.outputTokens ?? raw.output_tokens
    ?? raw.completionTokens ?? raw.completion_tokens;
  const cacheRead = raw.cacheRead ?? raw.cache_read ?? raw.cache_read_input_tokens;
  const cacheWrite = raw.cacheWrite ?? raw.cache_write ?? raw.cache_creation_input_toke
  const total = raw.total ?? raw.totalTokens ?? raw.total_tokens;
  return { input, output, cacheRead, cacheWrite, total };
}
```

每种字段都有 3-5 种可能的命名变体（驼峰、下划线、简称、全称）， `normalizeUsage()` 使用 `??` 链逐一尝试，确保无论哪种提供商的响应格式都能正确解析。

> **衍生解释**： `cacheRead` 和 `cacheWrite` 是 Anthropic 引入的"提示缓存"（Prompt Caching）特性对应的 token 类型。当系统提示（system prompt）被缓存后，后续请求读取缓存的 token 费用远低于常规输入 token。 `cacheWrite` 是首次写入缓存的 token 数， `cacheRead` 是后续命中缓存的 token 数。

38.3.2 会话级成本计算

`loadSessionCostSummary()` （ `src/infra/session-cost-usage.ts` ）从会话的 transcript 文件中逐行解析所有 LLM 调用记录，计算单个会话的总成本：

```
export type SessionCostSummary = CostUsageTotals & {
  messageCounts: SessionMessageCounts;      // 消息统计
  toolUsage: SessionToolUsage;              // 工具调用统计
  modelUsage: SessionModelUsage[];          // 按模型分组
  latency: SessionLatencyStats;             // 延迟统计
  dailyBreakdown: SessionDailyUsage[];      // 按日分组
  dailyMessageCounts: SessionDailyMessageCounts[];
  dailyLatency: SessionDailyLatency[];
  dailyModelUsage: SessionDailyModelUsage[];
};
```

解析过程使用 `readline` 逐行读取 JSONL 格式的 transcript 文件，避免一次性加载大文件到内存：

```
const rl = readline.createInterface({
  input: fs.createReadStream(sessionFile),
  crlfDelay: Infinity,
});
for await (const line of rl) {
  const entry = JSON.parse(line);
  // 提取 usage、cost、model、provider、duration 等
}
```

成本估算

当 API 响应不包含成本数据时（部分提供商不返回费用），OpenClaw 使用内置的模型定价表进行估算：

```
const cost = estimateUsageCost({
  usage: normalizedUsage,
  model: "claude-sonnet-4-20250514",
  provider: "anthropic",
});
```

定价表在 `src/utils/usage-format.ts` 中维护，覆盖了主流模型的输入/输出/缓存 token 单价。

38.3.3 跨会话聚合

Gateway 的 `sessions.usage` RPC 方法（`src/gateway/server-methods/usage.ts`，767 行）提供了跨会话的用量聚合查询。它支持按时间范围过滤和多维度聚合：

查询参数

参数	类型	默认值	说明
`startDate`	`YYYY-MM-DD`	30 天前	开始日期
`endDate`	`YYYY-MM-DD`	今天	结束日期
`days`	`number`	30	回溯天数（与日期互斥）
`limit`	`number`	50	返回会话数上限
`key`	`string`	—	查询单个会话
`includeContextWeight`	`boolean`	`false`	包含系统提示权重

聚合维度

返回的 `SessionsUsageAggregates` 包含以下维度：

```
export type SessionsUsageAggregates = {
  messages: SessionMessageCounts;        // 消息总计
  tools: SessionToolUsage;               // 工具调用总计
  byModel: SessionModelUsage[];          // 按模型分组（按成本降序）
  byProvider: SessionModelUsage[];       // 按提供商分组
  byAgent: Array<{ agentId; totals }>;   // 按 Agent 分组
  byChannel: Array<{ channel; totals }>; // 按频道分组
  latency?: SessionLatencyStats;         // 延迟统计（avg/p95/min/max）
  dailyLatency?: SessionDailyLatency[];  // 按日延迟
  modelDaily?: SessionDailyModelUsage[]; // 按模型按日
  daily: Array<{                         // 按日汇总
    date; tokens; cost; messages; toolCalls; errors
  }>;
};
```

缓存策略

跨会话聚合是计算密集型操作（需要读取多个 transcript 文件）。Gateway 使用 30 秒 TTL 的内存缓存：

```
const COST_USAGE_CACHE_TTL_MS = 30_000;
const costUsageCache = new Map<string, CostUsageCacheEntry>();

async function loadCostUsageSummaryCached(params) {
  const cacheKey = `${params.startMs}-${params.endMs}`;
  const cached = costUsageCache.get(cacheKey);
  if (cached?.summary && now - cached.updatedAt < COST_USAGE_CACHE_TTL_MS) {
    return cached.summary;  // 缓存命中
  }
  if (cached?.inFlight) {
    return cached.summary ?? await cached.inFlight;  // 去重并发请求
  }
  // 加载并缓存
}
```

与健康检查的缓存类似，这里也使用了 `inFlight` Promise 去重——多个客户端同时请求相同时间范围的数据时，只会执行一次实际计算。

38.3.4 提供商级用量追踪

`usage.status` RPC 返回各个 LLM 提供商的配额使用情况：

```
export type ProviderUsageSnapshot = {
  provider: UsageProviderId;   // "anthropic" | "openai-codex" | "google-gemini-cli" |
  displayName: string;         // 人类可读名称
  windows: UsageWindow[];      // 时间窗口内的用量
  plan?: string;               // 订阅计划（如 "pro"）
  error?: string;              // 查询错误
};

export type UsageWindow = {
  label: string;               // 如 "daily" / "monthly"
  usedPercent: number;          // 使用百分比 0-100
  resetAt?: number;             // 重置时间戳
};
```

OpenClaw 支持 8 种提供商的用量查询：Anthropic、GitHub Copilot、Google Gemini CLI、Google Antigravity、MiniMax、OpenAI Codex、Xiaomi、Zai。每种提供商有各自的 API 端点和认证方式，`loadProviderUsageSummary()` 统一封装了查询逻辑。

38.3.5 时序数据与日志

两个额外的 RPC 方法提供更细粒度的用量数据：

- **`sessions.usage.timeseries`**： 返回单个会话的用量时间序列（最多 200 个数据点），适用于控制台 UI 中绘制用量趋势图
- **`sessions.usage.logs`**： 返回单个会话的原始 LLM 调用日志（最多 1000 条），包含每次调用的模型、token 数、延迟、工具调用等详情

本节小结

1. **Token 归一化**（`normalizeUsage()`）解决了多提供商字段命名不一致的问题，将 5 种命名变体统一为标准的 `input/output/cacheRead/cacheWrite/total` 结构。
2. **会话级成本**通过逐行解析 JSONL transcript 文件计算，支持按日、按模型、按工具的多维度分解。缺少 API 成本数据时使用内置定价表估算。
3. **跨会话聚合**支持时间范围过滤和 6 种聚合维度（消息/工具/模型/提供商/Agent/频道），30 秒缓存 + Promise 去重优化性能。
4. **提供商配额**追踪支持 8 种 LLM 提供商的使用量和重置时间窗口查询。
5. 整套用量系统为控制台 UI 的"Usage"面板提供了数据支撑，帮助用户可视化理解 AI 调用的成本分布。